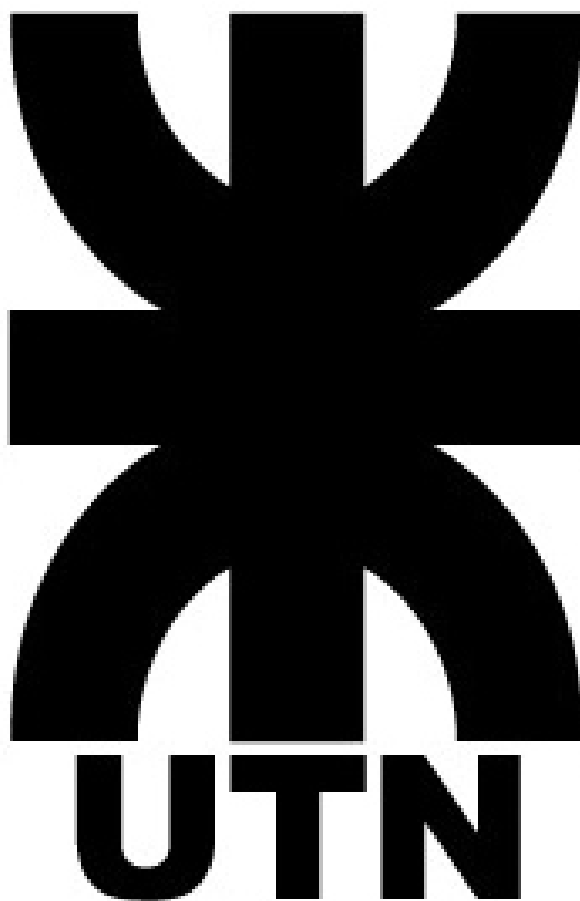


Programacion 3 – Trabajo Práctico Integrador “Food Store”

**Tecnicatura Universitaria en Programacion
Universidad Tecnologica Nacional**



Materia: Programacion 3

Estudiante: Gragera Lucas Ignacio

Legajo / Registro: 43524123

Profesor/es: Matias Santiago Torres, Enferrel Ariel, Cinthia Rigoni y Alberto Cortez

Junio 2026

Índice

1. Marco Teórico	3
1.1 Ecosistema y Tecnologías Utilizadas	3
2. Decisiones Técnicas y Arquitectura	4
2.1 Persistencia e Infraestructura de Datos	4
2.2 Arquitectura del Backend (JPA/Consola)	4
2.3 Diseño del Frontend y Control de Acceso)	4
2.4 Interfaces Principales	5
3. Dificultades y Soluciones	11
4. Bibliografía / Webgrafía	12

1. Marco Teórico

El sistema **Food Store** es una solución desarrollada con propósitos educativos que modela un flujo de negocio real de venta de alimentos, administración de catálogos y gestión de pedidos. El proyecto se compone de dos soluciones desacopladas: un módulo backend implementado en Java con persistencia relacional nativa y una aplicación frontend web moderna orientada a la experiencia del cliente y del administrador.

1.1 Tecnologías Utilizadas

Tecnología	Entorno	Propósito específico en el Proyecto
Java 21	Backend	Lenguaje base para el backend independiente. Utiliza características modernas y compilación orientada a Gradle.
JPA / Hibernate 6	Backend	Framework ORM (Jakarta Persistence) encargado de mapear la herencia de entidades y estructurar las tablas relacionales.
H2 Database	Backend / Datos	Motor relacional configurado en modo archivo (jdbc:h2:file:./data/jpa_db), asegurando persistencia histórica local.
Vite + TypeScript	Frontend	Ecosistema SPA ultrarrápido con tipado estricto para estructurar de manera limpia la interfaz del usuario.
Vanilla JS / DOM API	Frontend	Manipulación del árbol del DOM de forma nativa en los scripts asíncronos asociados a los archivos HTML sin sobrecarga de frameworks.
LocalStorage	Frontend	Mecanismo de persistencia local en el navegador para simular el almacenamiento de carritos, sesiones de usuarios y pedidos del cliente.

2. Decisiones Técnicas y Arquitectura

2.1 Persistencia e Infraestructura de Datos

El backend utiliza H2 en modo archivo persistido para garantizar la retención de los datos entre reinicios del sistema, empleando la estrategia update de Hibernate para la evolución automatizada del esquema a partir de los metadatos de las clases.

2.2 Arquitectura del Backend (JPA/Consola)

Se estructuró el código mediante un patrón de diseño orientado a Repositorios genéricos y herencia limpia:

- **Clase Base Mapeada (@MappedSuperclass):** Todas las entidades (Usuario, Producto, Categoria, Pedido, DetallePedido) heredan atributos comunes como id, eliminado (baja lógica) y createdAt a través de la superclase Base.
- **Repositorio Genérico (BaseRepository<T>):** Abstrae operaciones CRUD comunes mediante el ciclo de vida de un EntityManager centralizado en JPAUtil.
- **Transaccionalidad Atómica:** El alta de pedidos gestiona la coherencia del negocio de forma síncrona. Si el stock del producto es suficiente, reduce el stock, calcula subtotales, añade detalles al pedido y actualiza al usuario en una sola unidad de trabajo lógica (haciendo rollback ante excepciones).

2.3 Diseño del Frontend y Control de Acceso

La aplicación web simula las interacciones del negocio mediante interfaces modulares separadas en directorios por responsabilidades (auth, client, admin).

Nota sobre el nivel de seguridad del Frontend: Como se detalla en el README técnico, el control de accesos se gestiona mediante Guards basados en localStorage (checkAuthUser). Esta aproximación cumple fines estrictamente didácticos para ilustrar redirecciones en base a roles asignados (admin y client), advirtiéndole formalmente que no posee validez en entornos reales de producción por la mutabilidad del almacenamiento local desde las herramientas de desarrollo del navegador.

El carrito del cliente implementa lógica de cálculo asíncrono y persistencia local inmediata (carrito_compras), enlazando la confirmación de la orden al generar un UUID único (crypto.randomUUID()) y vinculándolo al correo electrónico del cliente logueado.

2.4 Interfaces Principales

Frontend (Aplicación Web Vite + TypeScript)

Login

Food Store

Iniciar Sesión

Email:

Contraseña:

Rol:

Seleccione un rol...▼

Ingresar

¿No tienes una cuenta? [Regístrate aquí](#)

Catalogo

Food Store

Inicio Mis Pedidos  Carrito

Panel Admin

Logout

Categorías

Todos los Productos

Pizzas

Hamburguesas

Bebidas sin Alcohol

Postres

Empanadas

Ensaladas

Bebidas con Alcohol

Golosinas

Buscar Productos:

Ordenar por...▼

Productos encontrados: 24

Nuestro Catálogo



Pizza Muzzarella

DISPONIBLE

Pizza clásica con salsa de tomate y muzzarella derretida

\$4600

Ver Detalle

Agregar



Pizza Napolitana

DISPONIBLE

Pizza con rodajas de tomate fresco, ajo y albahaca

\$5200

Ver Detalle

Agregar



Pizza Especial 4 Quesos

DISPONIBLE

Muzzarella, provolone, roquefort y parmesano

\$6800

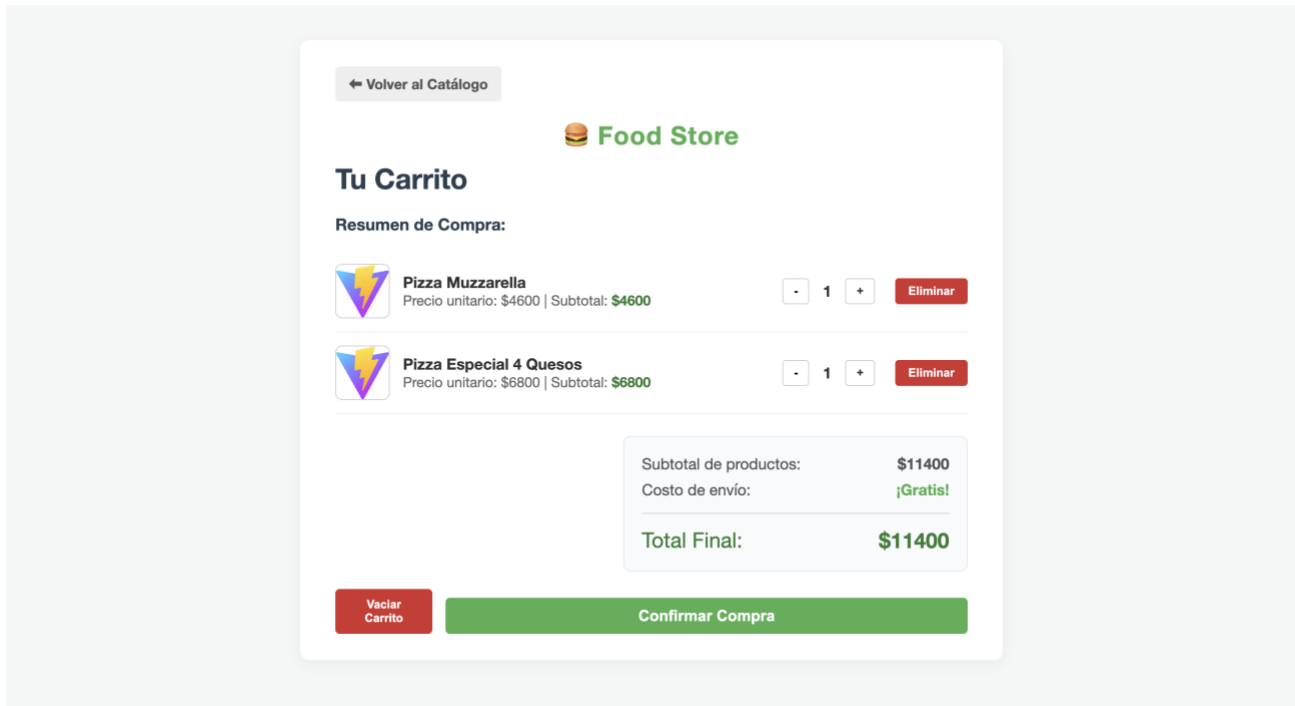
Ver Detalle

Agregar

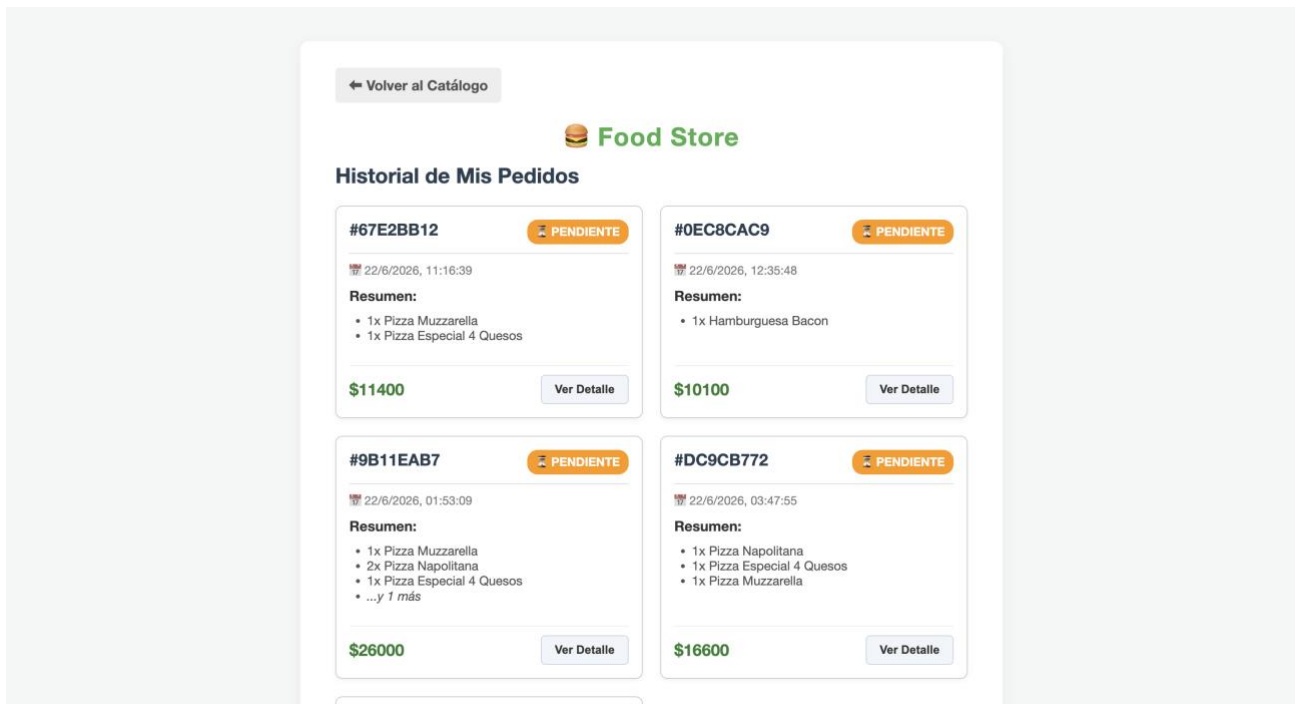
Gragera - Food Store - Documentación Académica y Técnica

5

Carrito



Pedidos



Panel de Administracion

 **Panel de Administración**

[Ir a la Tienda](#) [Salir](#)

Gestión de Categorías

Gestión de Productos

Gestión de Pedidos

 **Resumen Rápido**

 Ingresos Totales

\$23.900

 Pedidos Pendientes

9

 En Preparación

3

 Completados

2

 Cancelados

4

 **Resumen de Catálogo**

Total Categorías

8

Total Productos

24

Productos Activos

23

Total Pedidos Históricos

18

Gestion de Pedidos

 **Gestión de Pedidos**

Filtrar por estado:

Todos los pedidos

ID y Fecha	Cliente	Items	Total	Estado Actual	Acción
#67E2BB12 22/6/2026, 11:16:39	lucasgragera@gmail.com	2 producto(s)	\$11400	PENDIENTE	Pendiente Ver detalles
#7FAB46AA 22/6/2026, 04:38:03	lucasgragera@gmail.com	69 producto(s)	\$469200	CANCELADO	Cancelado Ver detalles
#DC9CB772 22/6/2026, 03:47:55	lucasgragera@gmail.com	3 producto(s)	\$16600	PENDIENTE	Pendiente Ver detalles
#9B11EAB7 22/6/2026, 01:53:09	lucasgragera@gmail.com	5 producto(s)	\$26000	PENDIENTE	Pendiente Ver detalles
#D89555E9 22/6/2026, 01:53:09	lucasgragera@gmail.com	5 producto(s)	\$26000	PENDIENTE	Pendiente Ver detalles

Gragera - Food Store - Documentación Académica y Técnica

7

Gestion de Categorías

Gestión de Categorías

+ Nueva Categoría

Guardar Categoría

ID	Imagen	Nombre	Descripción	Estado	Acciones
#1		Pizzas	Pizzas artesanales con masa fresca	Activa	Editar Dar de Baja
#2		Hamburguesas	Hamburguesas gourmet con ingredientes frescos	Activa	Editar Dar de Baja
#3		Bebidas sin Alcohol	Gaseosas, jugos y bebidas frías	Activa	Editar Dar de Baja
#4		Postres	Tortas, helados y dulces artesanales	Activa	Editar Dar de Baja
#5		Empanadas	Empanadas horneadas y fritas de distintos sabores	Activa	Editar Dar de Baja

Gestion de Productos

Gestión de Productos

+ Nuevo Producto

Guardar Producto

ID	Imagen	Nombre	Descripción	Categoría	Precio	Stock	Disponibilidad	Estado	Acciones
#1		Pizza Muzzarella	Pizza clásica con sals...	Pizzas	\$4600	18	Disponible	Activo	Editar Eliminar
#2		Pizza Napolitana	Pizza con rodajas de t...	Pizzas	\$5200	11	Disponible	Activo	Editar Eliminar
#3		Pizza Especial 4 Quesos	Muzzarella, provolone...	Pizzas	\$6800	29	Disponible	Activo	Editar Eliminar
#4		Hamburguesa Clásica	Medallón de carne, le...	Hamburguesas	\$3800	30	Disponible	Activo	Editar Eliminar
#5		Hamburguesa BBQ Bacon	Doble medallón, baco...	Hamburguesas	\$5100	25	Disponible	Activo	Editar Eliminar

Backend (Sistema de Consola Java 21 + JPA)

Alta de una categoria

```
===== FOOD STORE – MENÚ PRINCIPAL =====
1. Gestionar Categorías
2. Gestionar Productos
3. Gestionar Usuarios
4. Gestionar Pedidos
5. Reportes
0. Salir
Opción: 1

--- GESTIÓN DE CATEGORÍAS ---
1. Alta
2. Modificar
3. Baja lógica
4. Listado
0. Volver
Opción: 1
Nombre (obligatorio): Bebidas Alcoholicas
Descripción: Bebidas con contenido de alcohol
Categoría guardada con ID: 4
```

Dar de baja una categoria

```
--- GESTIÓN DE CATEGORÍAS ---
1. Alta
2. Modificar
3. Baja lógica
4. Listado
0. Volver
Opción: 3
ID de la categoría a dar de baja: 4
Categoría 'Bebidas Alcoholicas' dada de baja correctamente.

--- GESTIÓN DE CATEGORÍAS ---
1. Alta
2. Modificar
3. Baja lógica
4. Listado
0. Volver
Opción: 4

--- LISTADO DE CATEGORÍAS ACTIVAS ---
ID: 1 | Nombre: Pizzas | Descripción: Pizzas a la piedra
ID: 2 | Nombre: Pizza | Descripción: Pizza a la piedra
ID: 3 | Nombre: 0 | Descripción: 0
```

Alta de un producto

===== FOOD STORE - MENÚ PRINCIPAL =====

- 1. Gestionar Categorías
- 2. Gestionar Productos
- 3. Gestionar Usuarios
- 4. Gestionar Pedidos
- 5. Reportes
- 0. Salir

Opción: 2

--- GESTIÓN DE PRODUCTOS ---

- 1. Alta
- 2. Modificar
- 3. Baja lógica
- 4. Listado
- 0. Volver

Opción: 1

--- LISTADO DE CATEGORÍAS ACTIVAS ---

ID: 1 | Nombre: Pizzas | Descripción: Pizzas a la piedra

ID: 2 | Nombre: Pizza | Descripción: Pizza a la piedra

ID de la categoría a asignar: 1

Nombre (obligatorio): Pizza Napolitana

Descripción: Pizza Napolitana 8 porciones

Precio: 14000

Stock inicial: 11

Imagen (URL/Nombre):

Producto guardado y asociado a la categoría: Pizzas

--- GESTIÓN DE PRODUCTOS ---

- 1. Alta
- 2. Modificar
- 3. Baja lógica
- 4. Listado
- 0. Volver

Opción: 4

--- LISTADO DE PRODUCTOS ACTIVOS ---

ID: 1 | Nombre: Muzzarella | Precio: \$5000.00 | Stock: 6 | Disponible: Sí

ID: 2 | Nombre: 3 | Precio: \$3.00 | Stock: 3 | Disponible: Sí

ID: 3 | Nombre: Pizza Napolitana | Precio: \$14000.00 | Stock: 11 | Disponible: Sí

3. Dificultades y Soluciones

Durante los ciclos de desarrollo se presentaron desafíos lógicos que se resolvieron aplicando buenas prácticas de diseño de software:

Desafíos de Concurrencia y Persistencia Atómica

Problema: Durante el alta de pedidos por consola, la lectura fraccionada del inventario permitía la creación de órdenes de compra que superaban el stock físico real del almacén relacional, rompiendo la coherencia de negocio.

Solución: Se centralizó la lógica operativa del pedido vinculándola a un entorno transaccional explícito coordinado por EntityTransaction. Mediante una inspección en vivo (`em.find()`), el sistema valida inmediatamente que la cantidad solicitada sea menor o igual al stock del objeto gestionado; ante una insuficiencia, se interrumpe el ciclo y se ejecuta un `rollback()` atómico de la transacción, preservando el estado original de la base de datos de forma segura.

Resoluciones del Ciclo de Vida en Módulos ES

Problema: Al estructurar la interactividad de las vistas administrativas utilizando módulos nativos de TypeScript empaquetados por Vite, las directivas tradicionales incrustadas en línea dentro del HTML (como `onclick` o `onchange`) devolvían errores de referencia (`is not defined`) debido a que el empaquetador aísla el alcance del código para evitar colisiones globales.

Solución: Se rompió el aislamiento de ámbito controlado de manera intencional y segura, exponiendo los manejadores de eventos críticos hacia la ventana global del navegador mediante moldes explícitos al objeto global de ejecución del entorno de desarrollo: `(window as any).editarProducto = ...` y `(window as any).cambiarEstado = ....`

Quiebre de Sincronización del Catálogo en Servidor Local (localhost)

Problema: Al inicializar el entorno local de desarrollo de manera desacoplada, los scripts de renderizado del catálogo del cliente (`catalogo.ts`) y de la gestión administrativa (`products.ts`) presentaban fallas de sincronización. Las mutaciones ejecutadas al confirmar pedidos o dar de alta productos no impactaban uniformemente, lo que provocaba que una pantalla mostrara desabastecimiento total mientras la otra continuaba visualizando existencias estáticas, derivando en un desborde visual del contenedor y discrepancias de estado severas, como también descincronización de categorías.

Solución: El error se identificó en el uso inconsistente de los punteros de acceso clave-valor del almacenamiento del cliente local y en la persistencia de estilos en línea rígidos en el contenedor del DOM. Se homogeneizaron de manera estricta todas las claves del software bajo la estructura unificada `"productos_db"` y `"categorias_db"`, entrelazando las cargas iniciales a una evaluación asíncrona de las semillas del sistema. En paralelo, se removieron los estilos rígidos inline de la grilla reemplazándolos por un diseño fluido basado en CSS Grid (`repeat(auto-fill, minmax(220px, 1fr))`) combinado con la propiedad de empuje `margin-top: auto` en las botoneras. Esto garantizó que, de forma unívoca en

cualquier puerto local, las mutaciones de stock, variaciones de los tableros del administrador y las dimensiones de las tarjetas se recalculen y alineen en tiempo real de manera perfectamente simétrica.

Control de Recursión Cíclica en Relaciones Bilaterales

Problema: La estrecha asociación relacional entre colecciones unidireccionales de JPA producía desbordamientos de pila de ejecución de memoria física (StackOverflowError) al intentar volcar o mapear las entidades por consola mediante métodos automáticos de cadena de texto.

Solución: Se aplicaron directivas explícitas de Lombok a nivel de clase utilizando @ToString(callSuper = true, exclude = {...}). De esta manera, se omitió deliberadamente la resolución de colecciones anidadas y bidireccionales dentro del método de impresión estructurado, erradicando los ciclos infinitos de manera limpia.

4. Bibliografía / Webgrafía

Documentación técnica y académica consultada:

- **Java 21 & 22 Documentation:** <https://docs.oracle.com/en/java/javase/>
- **Hibernate ORM 6.4 User Guide:** <https://hibernate.org/orm/documentation/6.4/>
- **Vite Bundler Guide:** <https://vitejs.dev/>
- **H2 Database Engine Manual:** <https://www.h2database.com/>
- **TypeScript Deep Dive & Reference:** <https://www.typescriptlang.org/docs/>